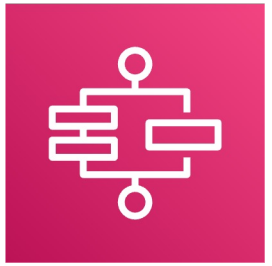
Service Communication

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS Step Functions



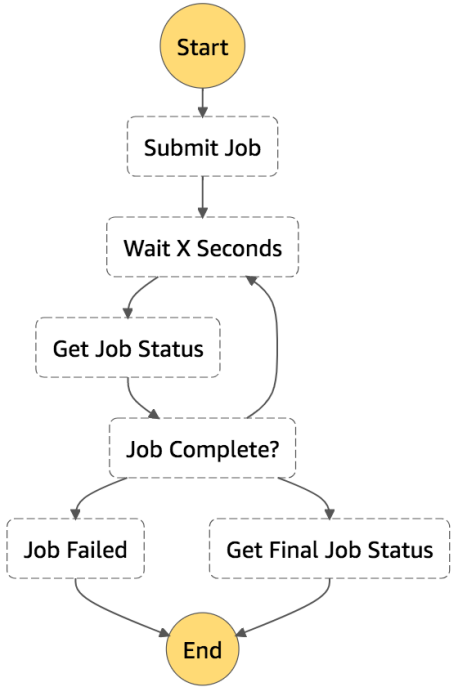
- Build serverless visual workflow to orchestrate your Lambda functions
- Represent flow as a JSON **state machine**
- Features: sequence, parallel, conditions, timeouts, error handling…
- Maximum execution time of 1 year
- Possibility to implement human approval feature

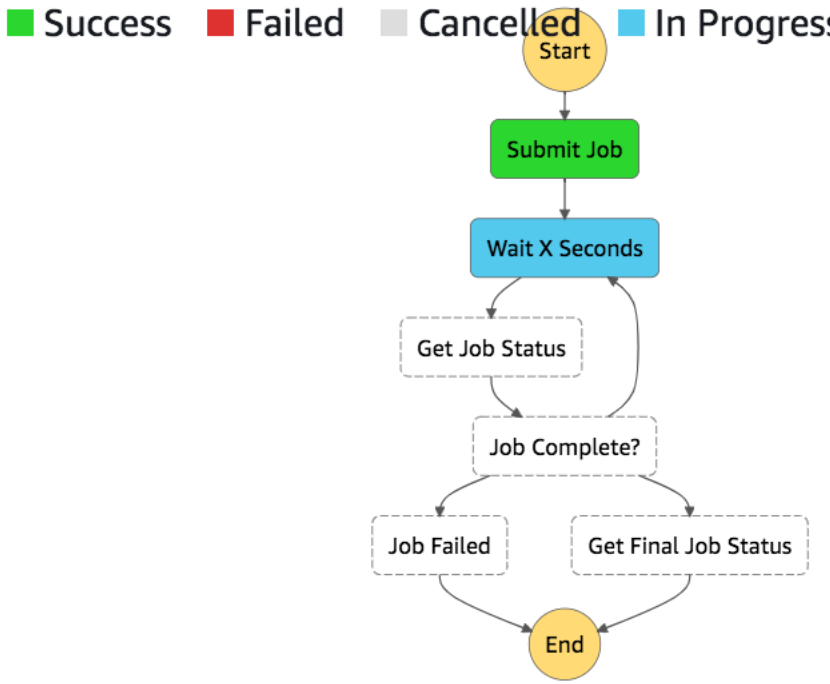
- If you chain Lambda functions using Step Functions, be mindful of the added latency to pass the calls.

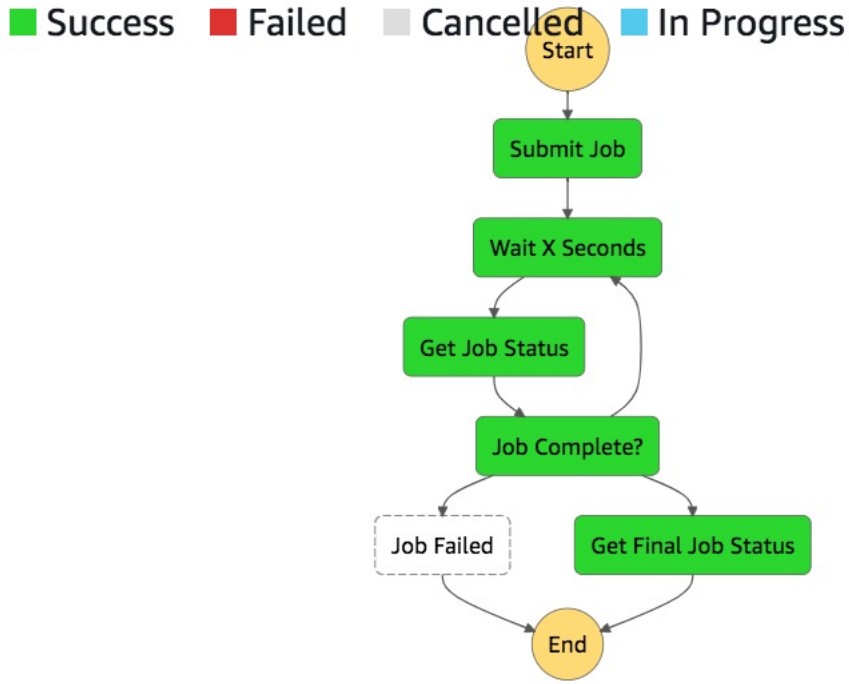
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Visual workflow in Step Functions







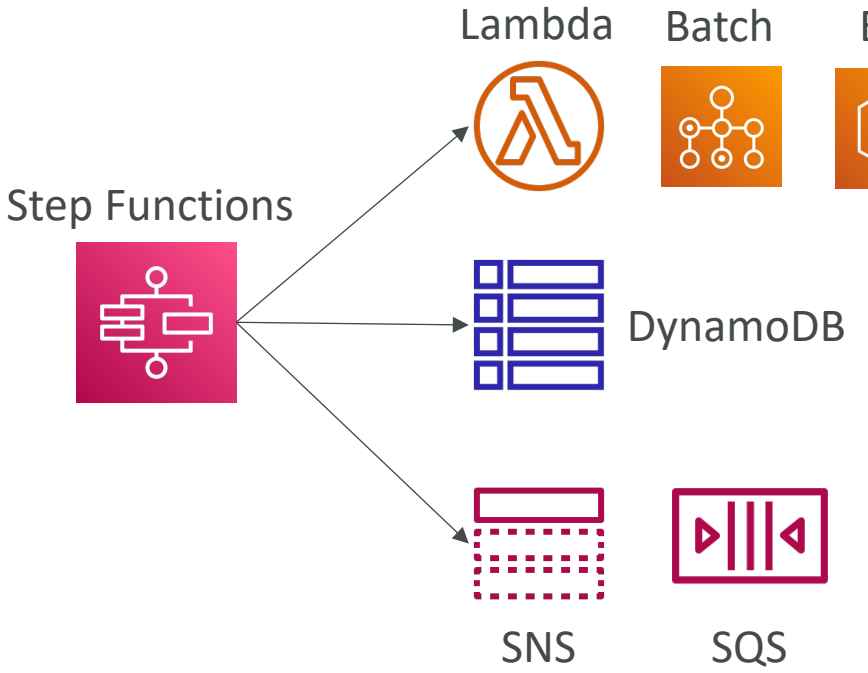
NOT FOR DISTRIBU

jmulus.com

© Stephane Maarek

Step Function Integrations

- Optimized Integrations
  - Can invoke a **Lambda function**
  - Run an AWS Batch job
  - Run an ECS task and wait for it to complete
  - Insert an item from DynamoDB
  - Publish message to SNS, SQS
  - Launch an EMR, Glue, or SageMaker jobs
  - Launch another Step Function workflow…
- AWS SDK Integrations
  - Access 200+ AWS services from your State Machine
  - Works like standard AWS SDK API call

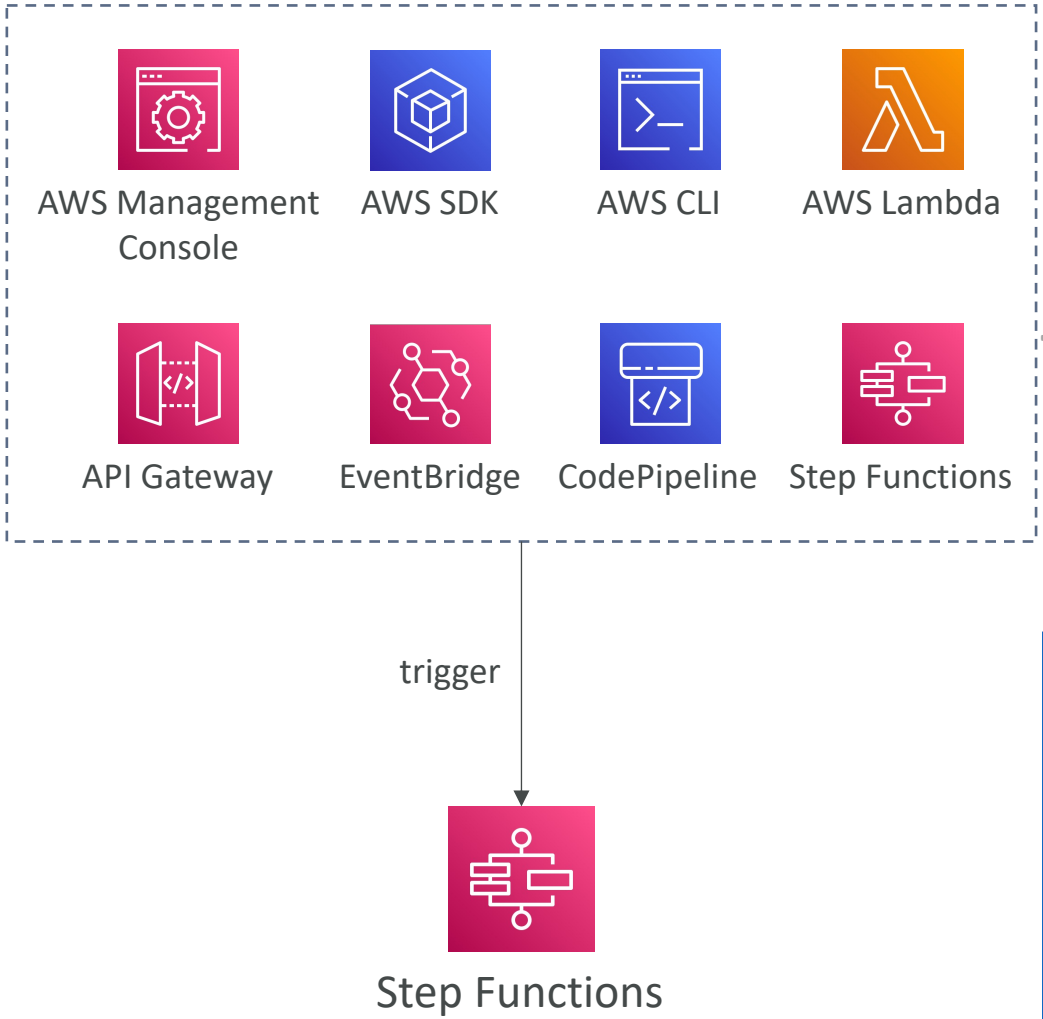


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Step Functions Workflow Triggers

- You can invoke a Step Function Workflow (State Machine) using:
  - AWS Management Console
  - AWS SDK (**StartExecution** API call)
  - AWS CLI (**start-execution**)
  - AWS Lambda (**StartExecution** API call)
  - API Gateway
  - EventBridge
  - CodePipeline
  - Step Functions

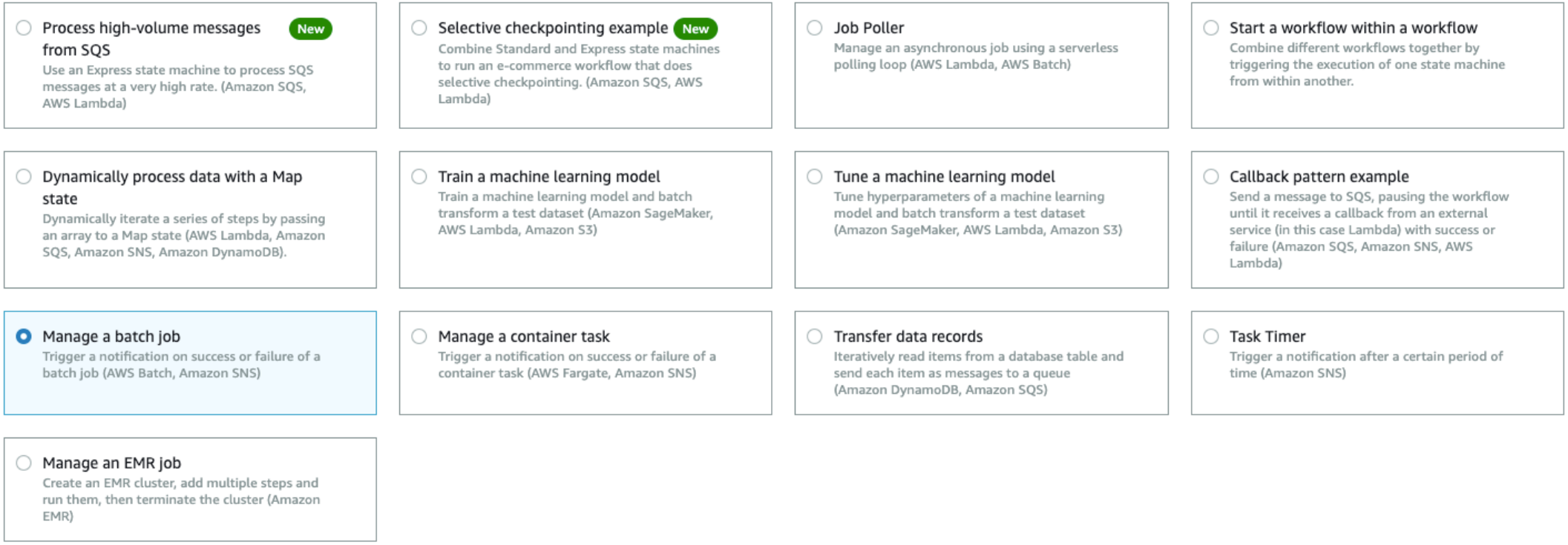


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Step Functions – Sample Projects

- https://console.aws.amazon.com/states/home?region=us-east-1#/sampleProjects



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Step Functions – Tasks

- Lambda Tasks:
  - Invoke a Lambda function
- Activity Tasks:
  - Activity worker (HTTP), EC2 Instances, mobile device, on premise DC
  - They poll the Step functions service
- Service Tasks:
  - Connect to a supported AWS service
  - Lambda function, ECS Task, Fargate, DynamoDB, Batch job, SNS topic, SQS queue
- Wait Task:
  - To wait for a duration or until a timestamp

- Note: Step Functions does not integrate natively with AWS Mechanical Turk

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Step Functions – Standard vs. Express

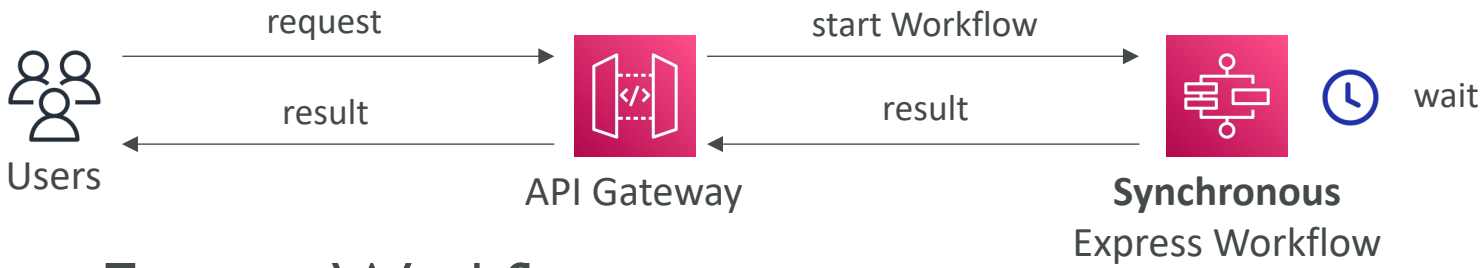
	Standard Workflows	Express Workflows
Maximum duration	1 year.	5 minutes.
Supported execution start rate	Over 2,000 per second	Over 100,000 per second
Supported state transition rate	Over 4,000 per second per account	Nearly unlimited
Pricing	Priced per state transition. A state transition is counted each time a step in your execution is completed (more expensive)	Priced by the number of executions you run, their duration, and memory consumption (cheaper)
Execution history	Executions can be listed and described with Step Functions APIs, and visually debugged through the console. They can also be inspected in CloudWatch Logs by enabling logging on your state machine.	Executions can be inspected in CloudWatch Logs by enabling logging on your state machine.
Execution semantics	Exactly-once workflow execution.	At-least-once workflow execution.

© Stephane Maarek

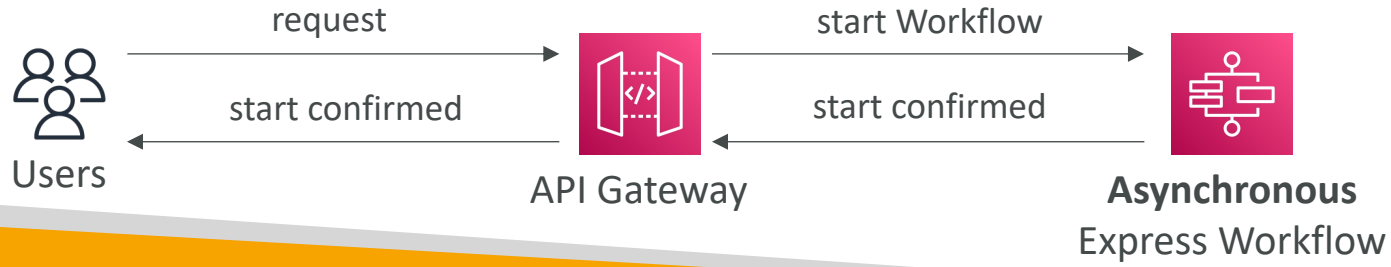
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Step Functions – Express Workflow – Synchronous vs. Asynchronous

- **Synchronous Express Workflows**
  - Wait until the Workflow completes, then return the result
  - Examples: orchestrate microservices, handle errors, retries, parallel tasks, …



- **Asynchronous Express Workflows**
  - Doesn't for the Workflow to complete
  - Examples: Workflows that don't require immediate response, messaging, …

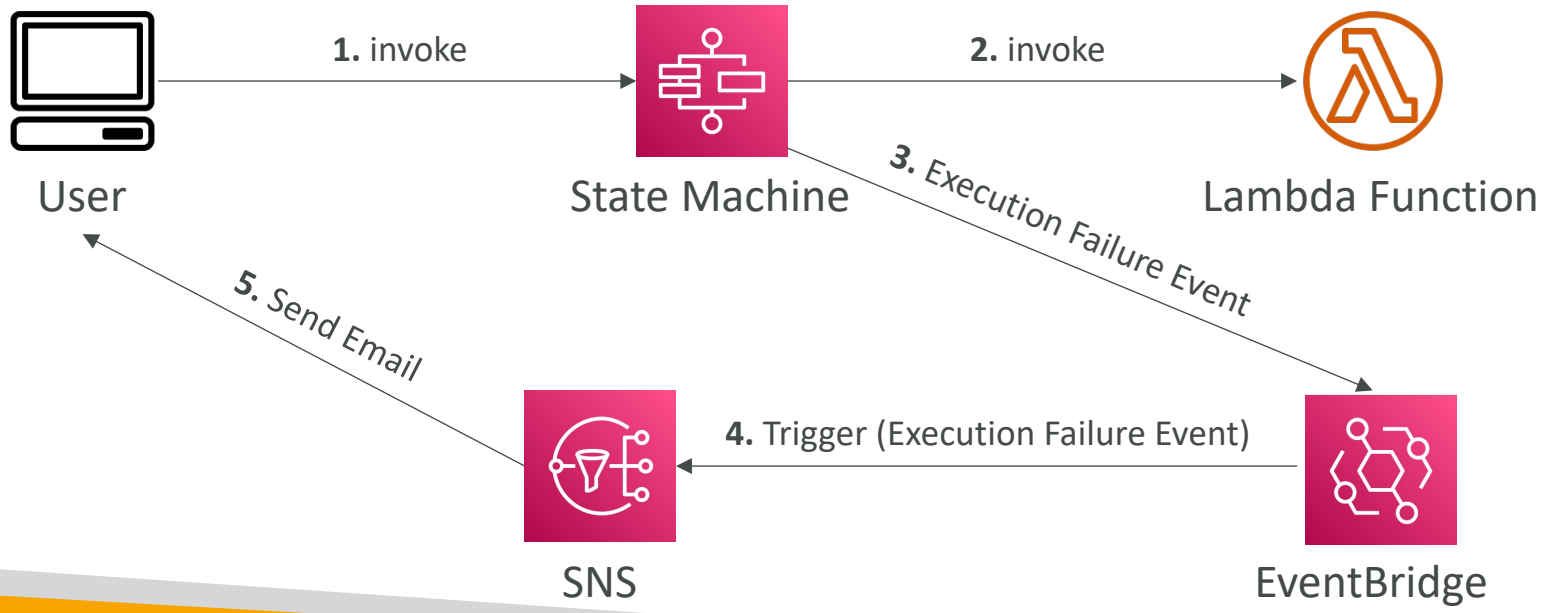


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Step Functions – Error Handling

- You can enable error handling, retries, and add alerting to Step Function State Machine
- Example: set up EventBridge to alert via email if a State Machine execution fails

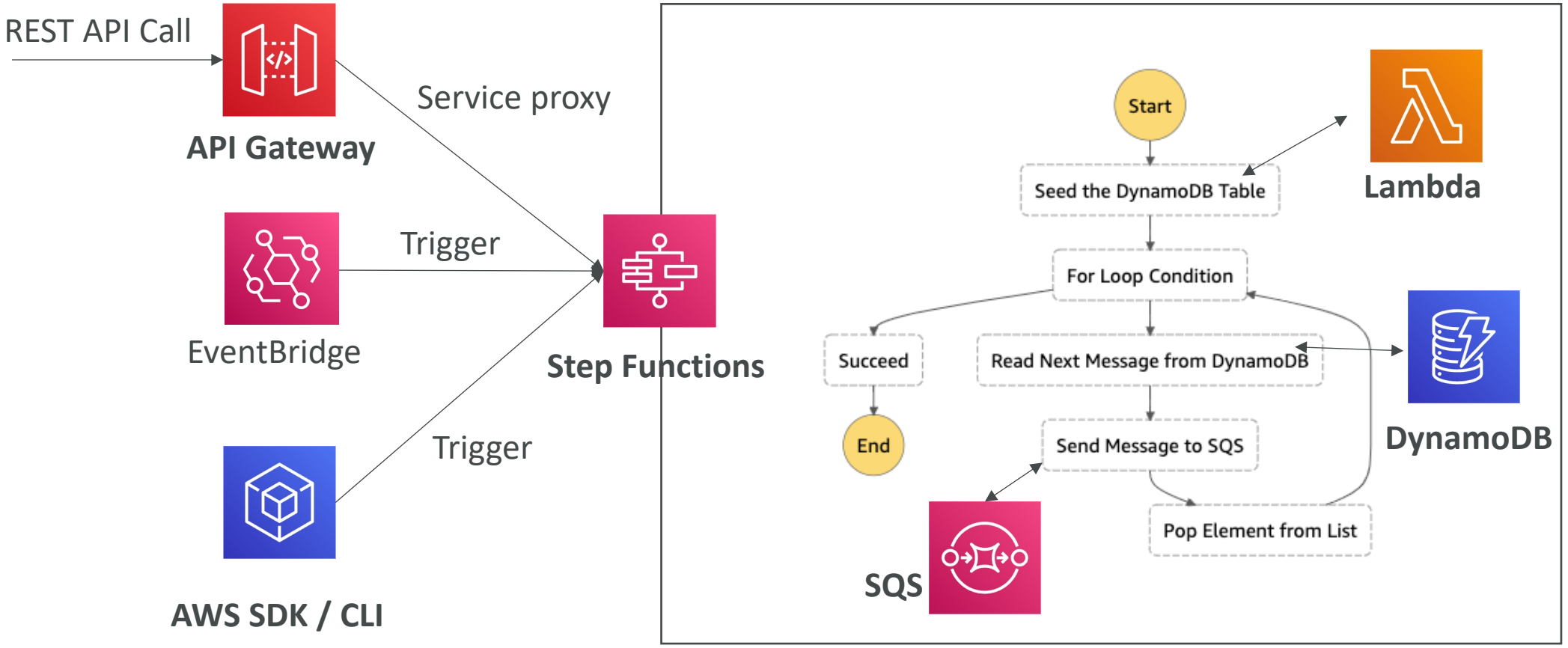



```
{
    "Retry": [
        {
            "ErrorEquals": ["States.ALL"],
            "IntervalSeconds": 3,
            "MaxAttempts": 2,
            "BackoffRate": 1.5
        }
    ]
}
```

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Step Functions – Solution Architecture



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon SQS



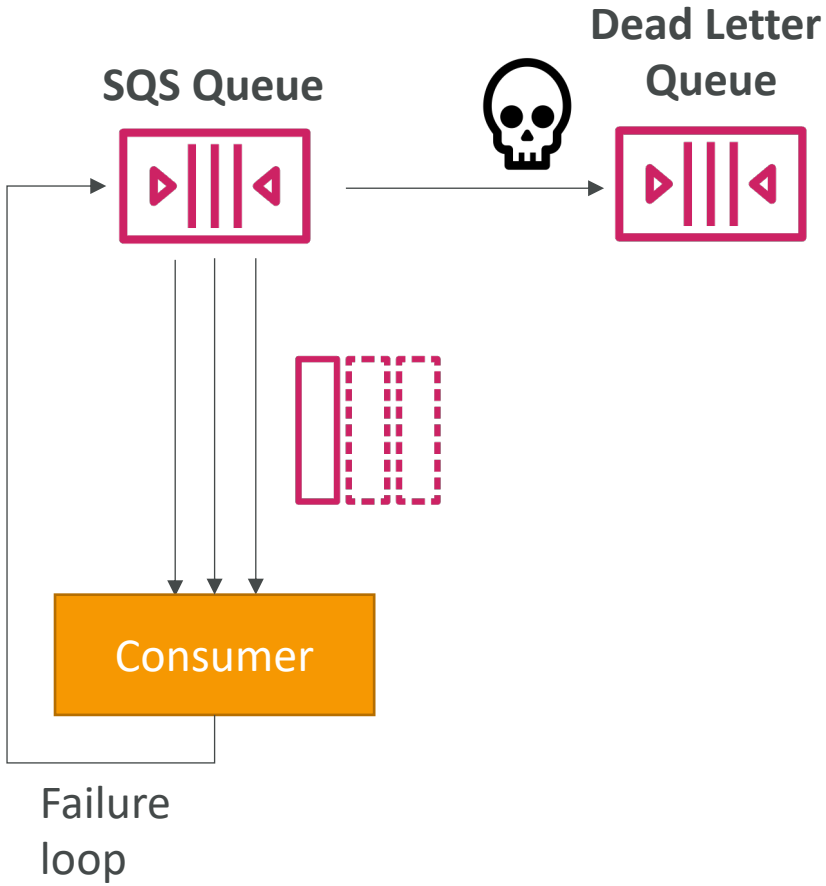
- Serverless, managed queue, integrated with IAM
- Can handle extreme scale, no provisioning required
- Used to **decouple** services
- Message size of max 1024 KB (use a pointer to S3 for large messages)
- Can be read from EC2 (optional ASG), Lambda
- SQS could be used as a write buffer for DynamoDB
- **SQS FIFO:**
  - receive messages in order they were sent
  - 300 messages/s without batching, 3000 /s with batching

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon SQS – Dead Letter Queue (DLQ)

- If a consumer fails to process a message within the Visibility Timeout…
  the message goes back to the queue!
- We can set a threshold of how many times a message can go back to the queue
- After the **MaximumReceives** threshold is exceeded, the message goes into a Dead Letter Queue (DLQ)
- Useful for debugging!
- **DLQ of a FIFO queue must also be a FIFO queue**
- **DLQ of a Standard queue must also be a Standard queue**
- Make sure to process the messages in the DLQ before they expire:
  - Good to set a retention of 14 days in the DLQ

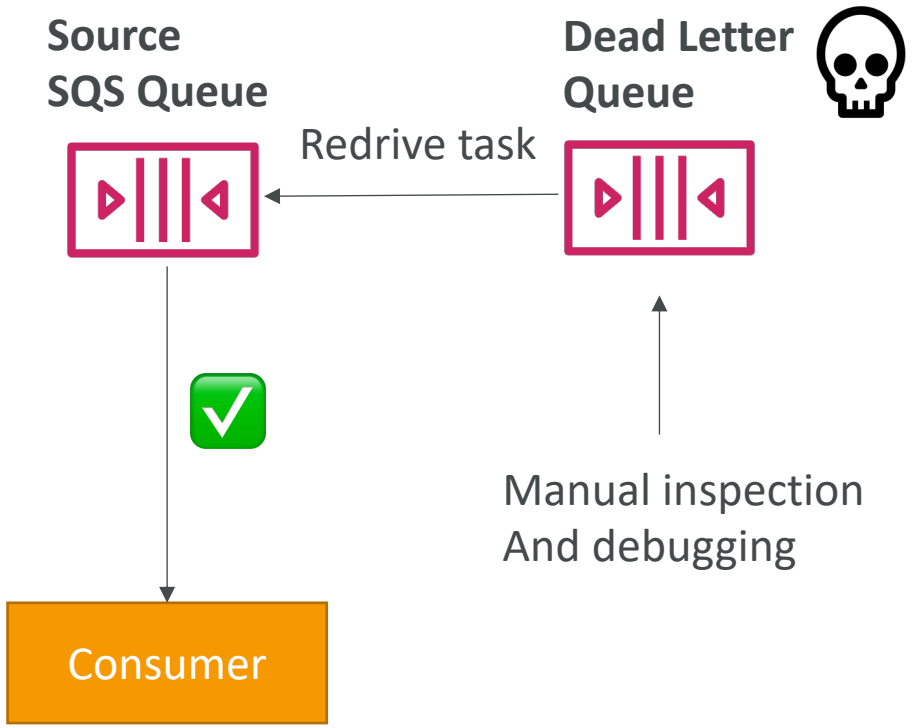


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SQS DLQ – Redrive to Source

- Feature to help consume messages in the DLQ to understand what is wrong with them
- When our code is fixed, we can redrive the messages from the DLQ back into the source queue (or any other queue) in batches without writing custom code

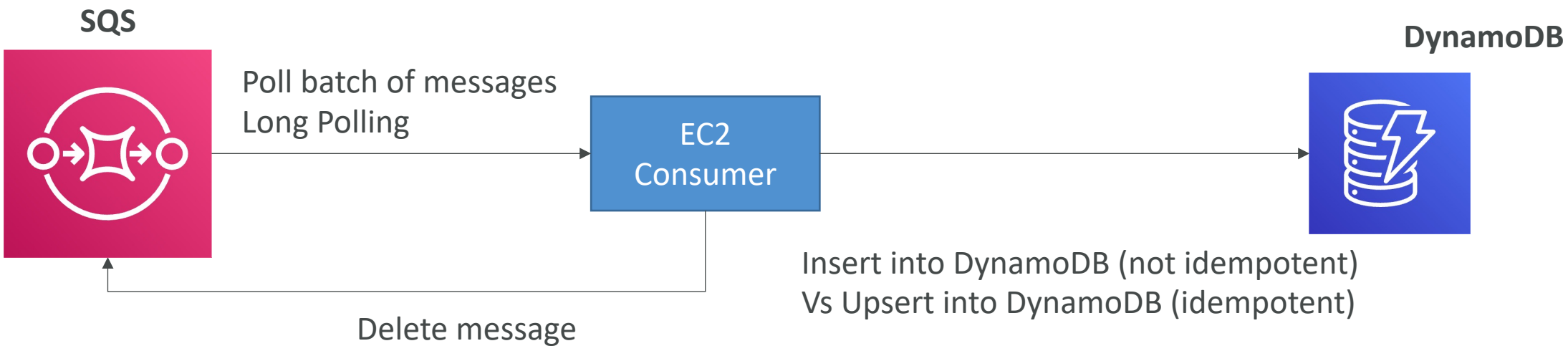


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SQS – Solution Architecture Idempotency

- Messages can be processed twice by consumer (in case of failures, timeouts, etc)
- To hedge against that problem, implement **idempotency** at the consumer level
- Means the same action done twice by the consumer won't duplicate the effect

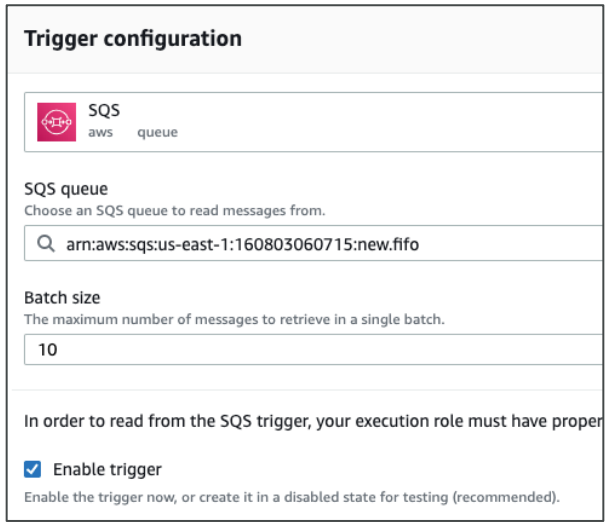


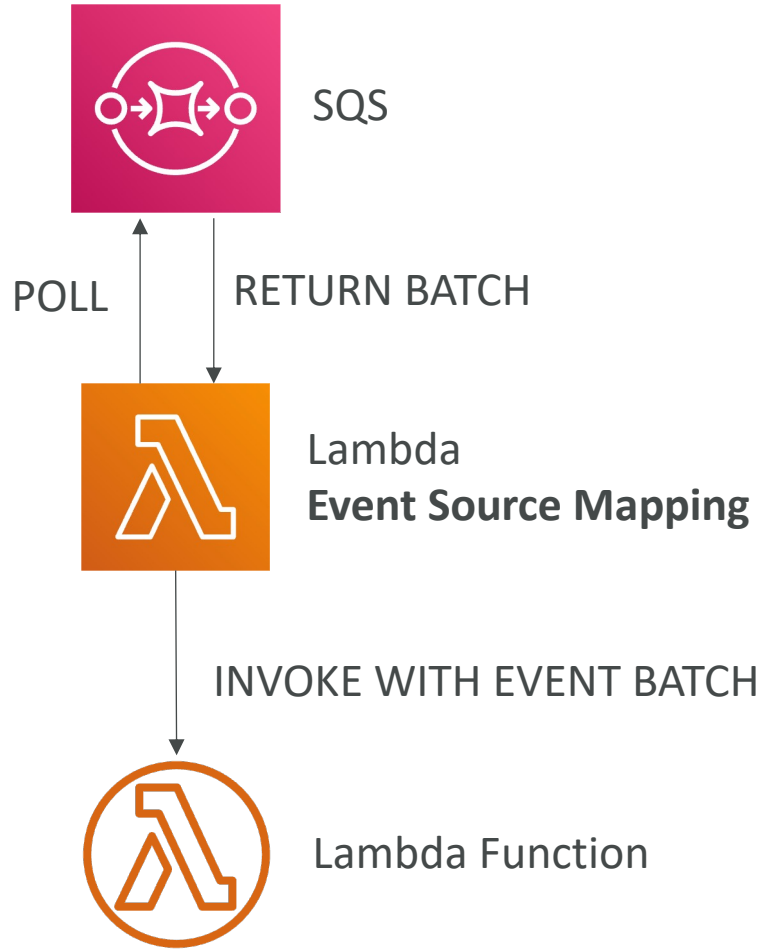
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Lambda – Event Source Mapping
SQS & SQS FIFO

- Event Source Mapping will poll SQS (Long Polling)
- Specify **batch size** (1-10 messages)
- Recommended: Set the queue visibility timeout to 6x the timeout of your Lambda function
- To use a DLQ
  - **set-up on the SQS queue, not Lambda** (DLQ for Lambda is only for async invocations)
  - Or use a Lambda destination for failures

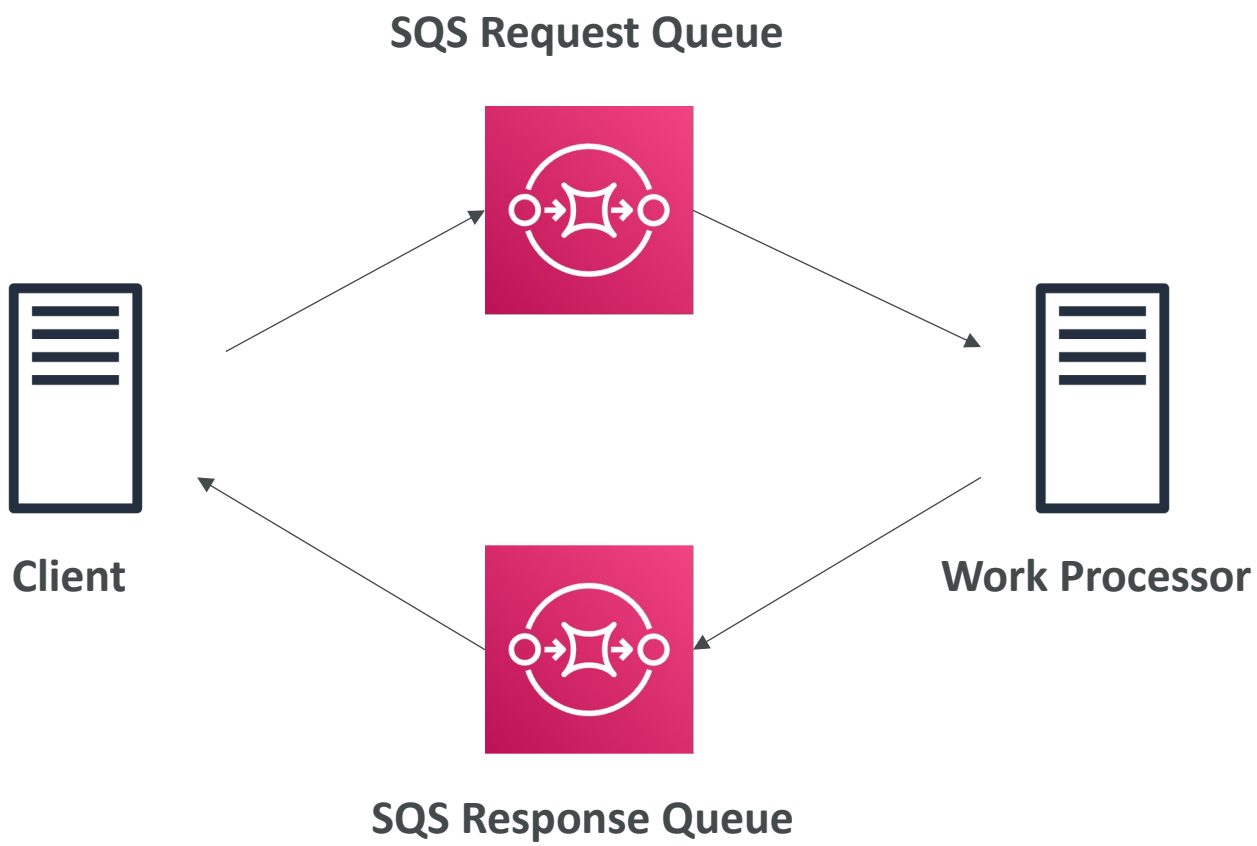




NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© Stephane Maarek

SQS - Solution Architecture
Request / Response queue (async)



- Decoupling
- Fault-Tolerance
- Load Balancing

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon MQ



- SQS, SNS are "cloud-native" services: proprietary protocols from AWS
- Traditional applications running from on-premises may use open protocols such as: MQTT, AMQP, STOMP, Openwire, WSS
- **When migrating to the cloud**, instead of re-engineering the application to use SQS and SNS, we can use Amazon MQ
- **Amazon MQ is a managed message broker service for**

RabbitMQ™



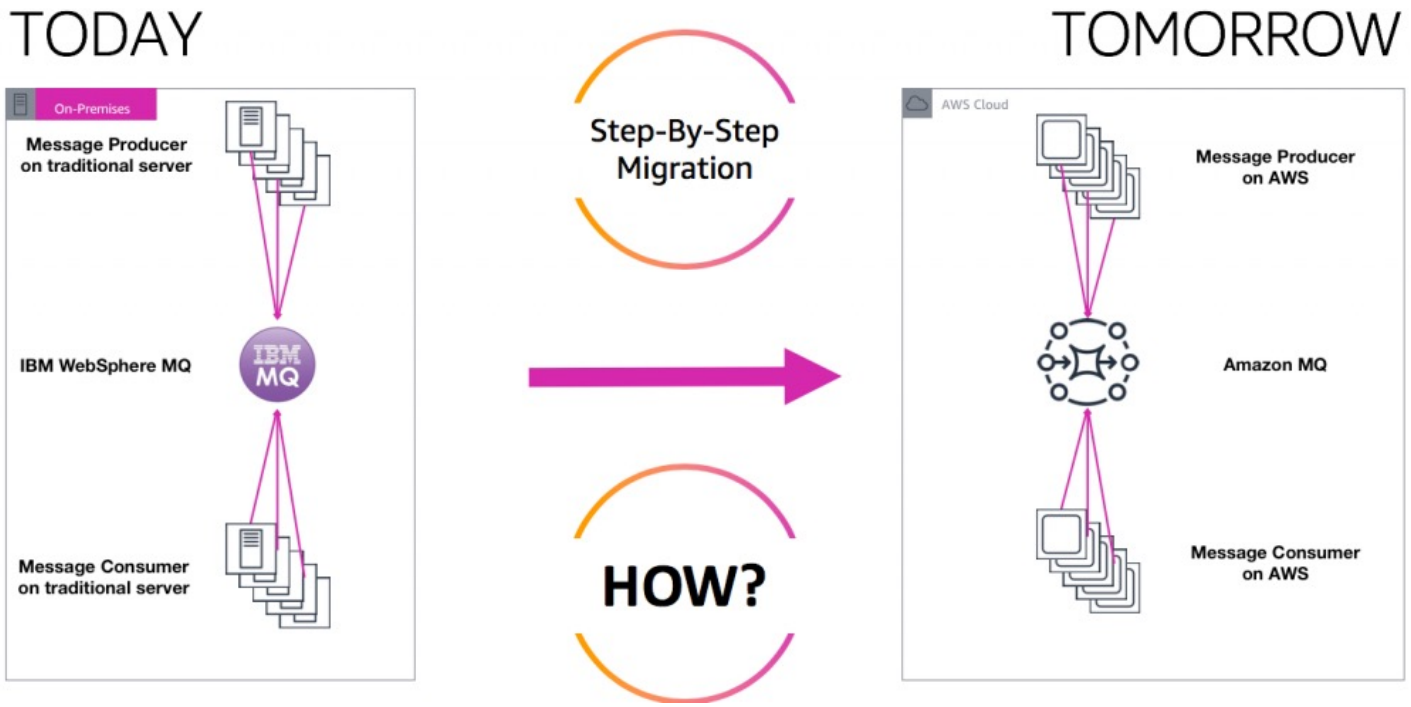
- Amazon MQ doesn't "scale" as much as SQS / SNS
- Amazon MQ runs on servers, can run in Multi-AZ with failover
- Amazon MQ has both queue feature (~SQS) and topic features (~SNS)

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon MQ – Re-platform

- IBM MQ, TIBCO EMS, Rabbit MQ, and Apache ActiveMQ can be migrated to Amazon MQ



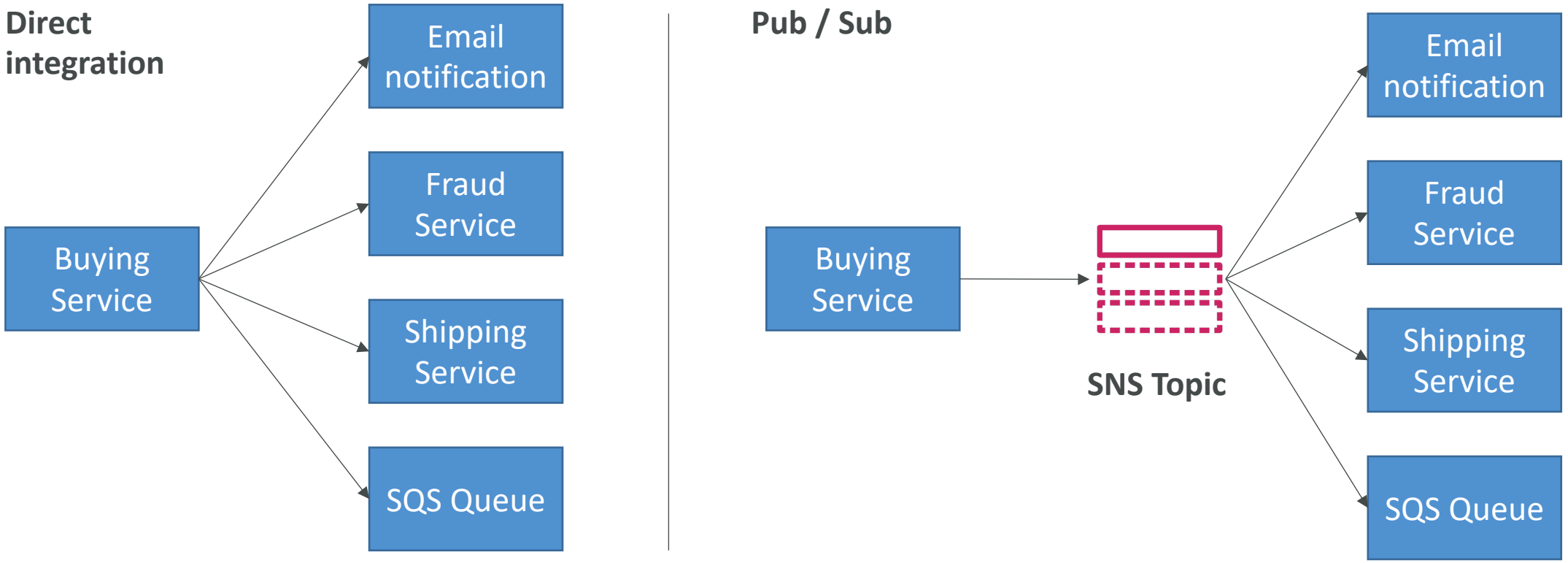
https://aws.amazon.com/blogs/compute/migrating-from-ibm-mq-to-amazon-mq-using-a-phased-approach/

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon SNS

- What if you want to send one message to many receivers?



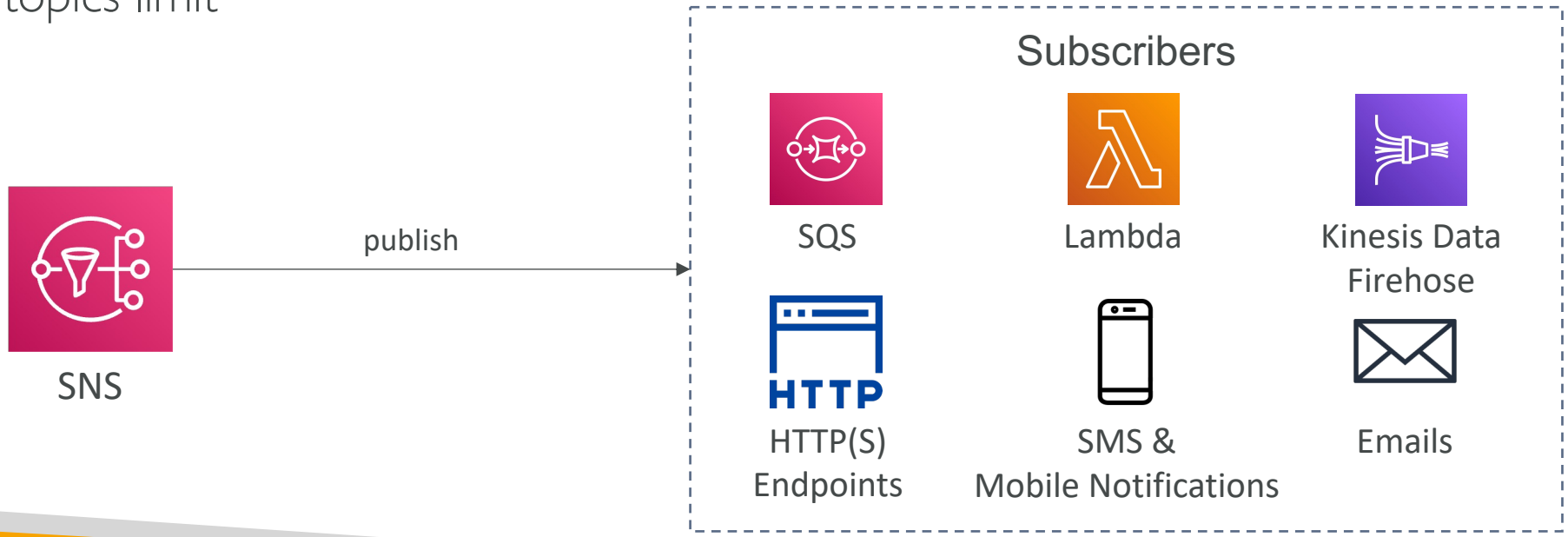
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon SNS



- The "event producer" only sends message to one SNS topic
- As many "event receivers" (subscriptions) as we want to listen to the SNS topic notifications
- Each subscriber to the topic will get all the messages (note: new feature to filter messages)
- Up to 12,500,000 subscriptions per topic
- 100,000 topics limit

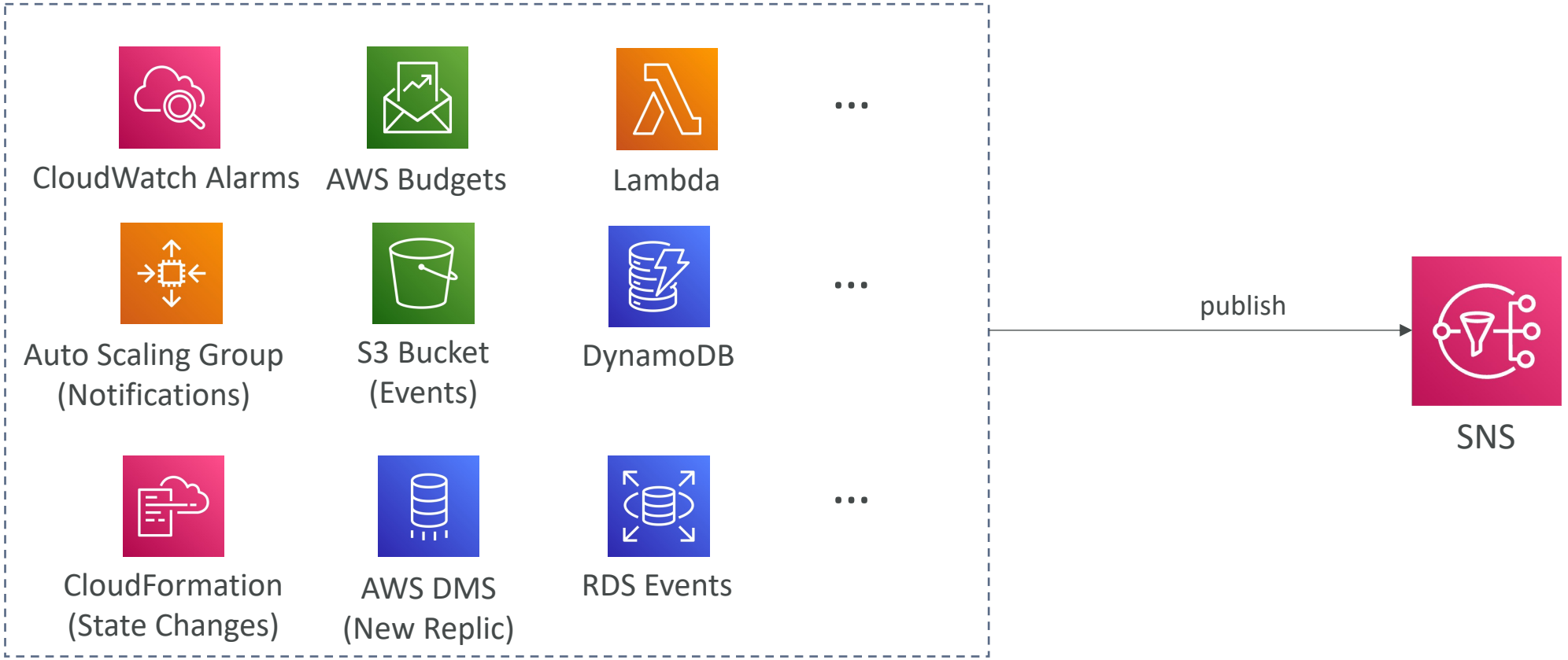


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SNS integrates with a lot of AWS services

- Many AWS services can send data directly to SNS for notifications



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon SNS – How to publish

- Topic Publish (using the SDK)
  - Create a topic
  - Create a subscription (or many)
  - Publish to the topic

- Direct Publish (for mobile apps SDK)
  - Create a platform application
  - Create a platform endpoint
  - Publish to the platform endpoint
  - Works with Google GCM, Apple APNS, Amazon ADM…

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

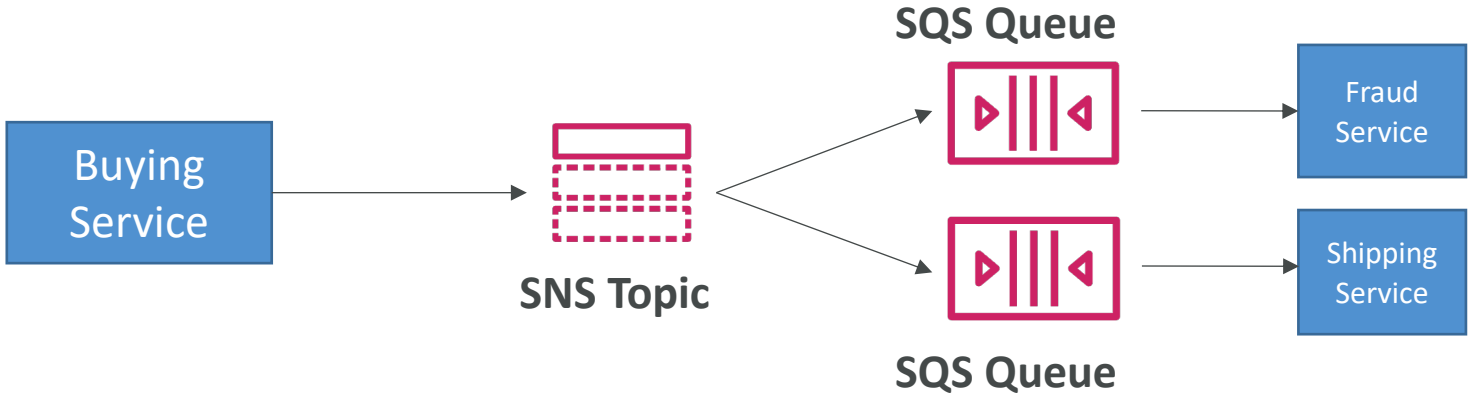
Amazon SNS – Security

- **Encryption:**
  - In-flight encryption using HTTPS API
  - At-rest encryption using KMS keys
  - Client-side encryption if the client wants to perform encryption/decryption itself
- **Access Controls:** IAM policies to regulate access to the SNS API
- **SNS Access Policies** (similar to S3 bucket policies)
  - Useful for cross-account access to SNS topics
  - Useful for allowing other services (S3…) to write to an SNS topic

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SNS + SQS: Fan Out



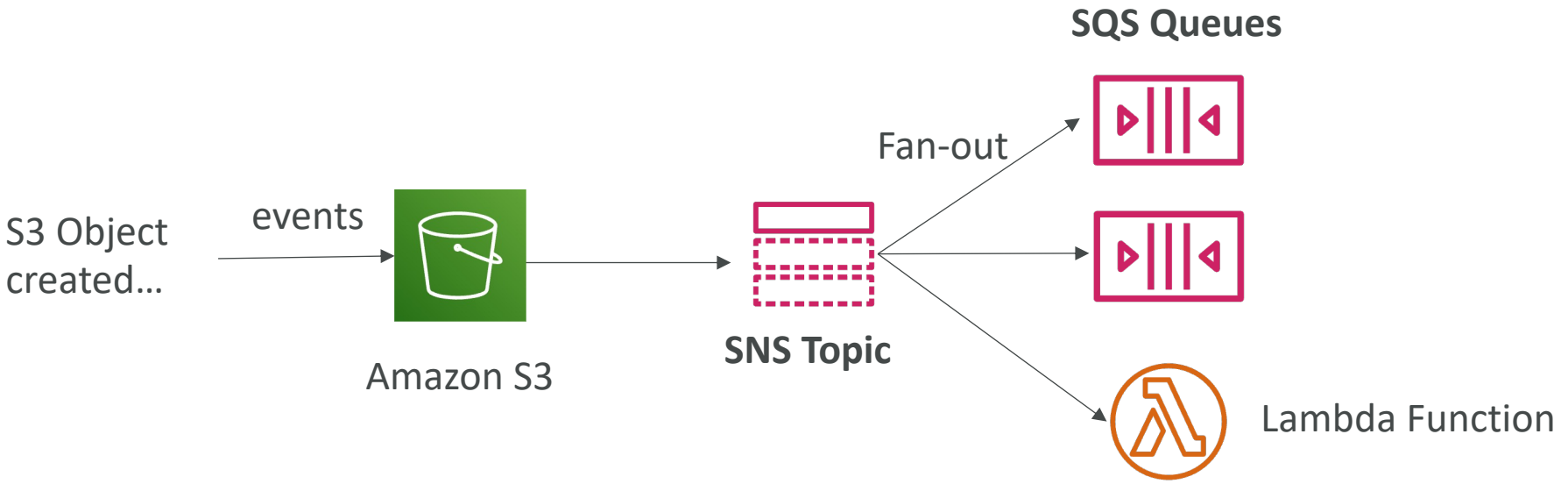
- Push once in SNS, receive in all SQS queues that are subscribers
- Fully decoupled, no data loss
- SQS allows for: data persistence, delayed processing and retries of work
- Ability to add more SQS subscribers over time
- Make sure your SQS queue **access policy** allows for SNS to write
- Cross-Region Delivery: works with SQS Queues in other regions

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Application: S3 Events to multiple queues

- For the same combination of: **event type** (e.g. object create) and **prefix** (e.g. images/) you can only have one S3 Event rule
- If you want to send the same S3 event to many SQS queues, use fan-out



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Application: SNS to Amazon S3 through Kinesis Data Firehose

- SNS can send to Kinesis and therefore we can have the following solutions architecture:

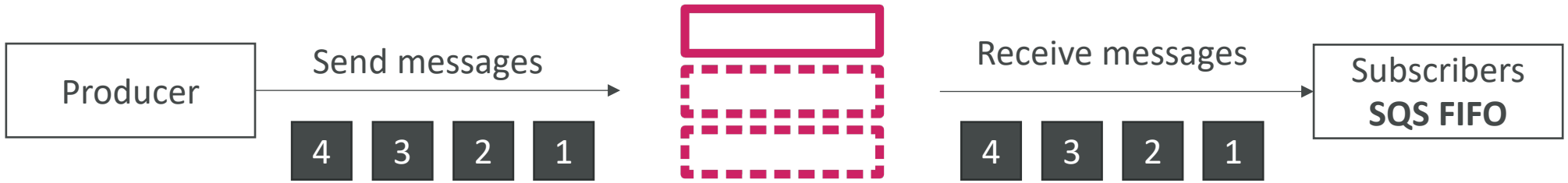


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon SNS – FIFO Topic

- FIFO = First In First Out (ordering of messages in the topic)



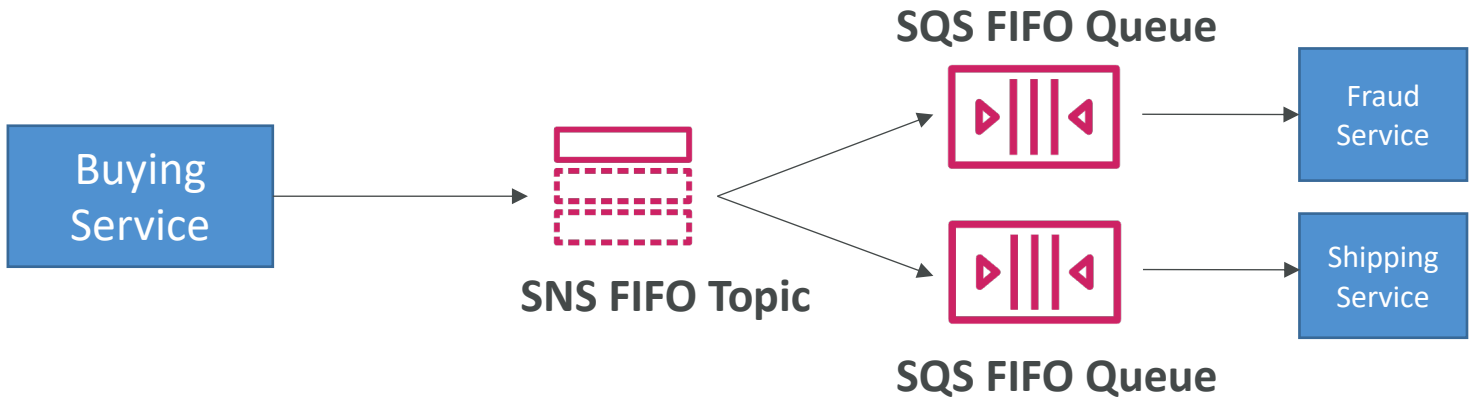
- Similar features as SQS FIFO:
  - **Ordering** by Message Group ID (all messages in the same group are ordered)
  - **Deduplication** using a Deduplication ID or Content Based Deduplication
- **Can have SQS Standard and FIFO queues as subscribers**
- Limited throughput (same throughput as SQS FIFO)

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SNS FIFO + SQS FIFO: Fan Out

- In case you need fan out + ordering + deduplication

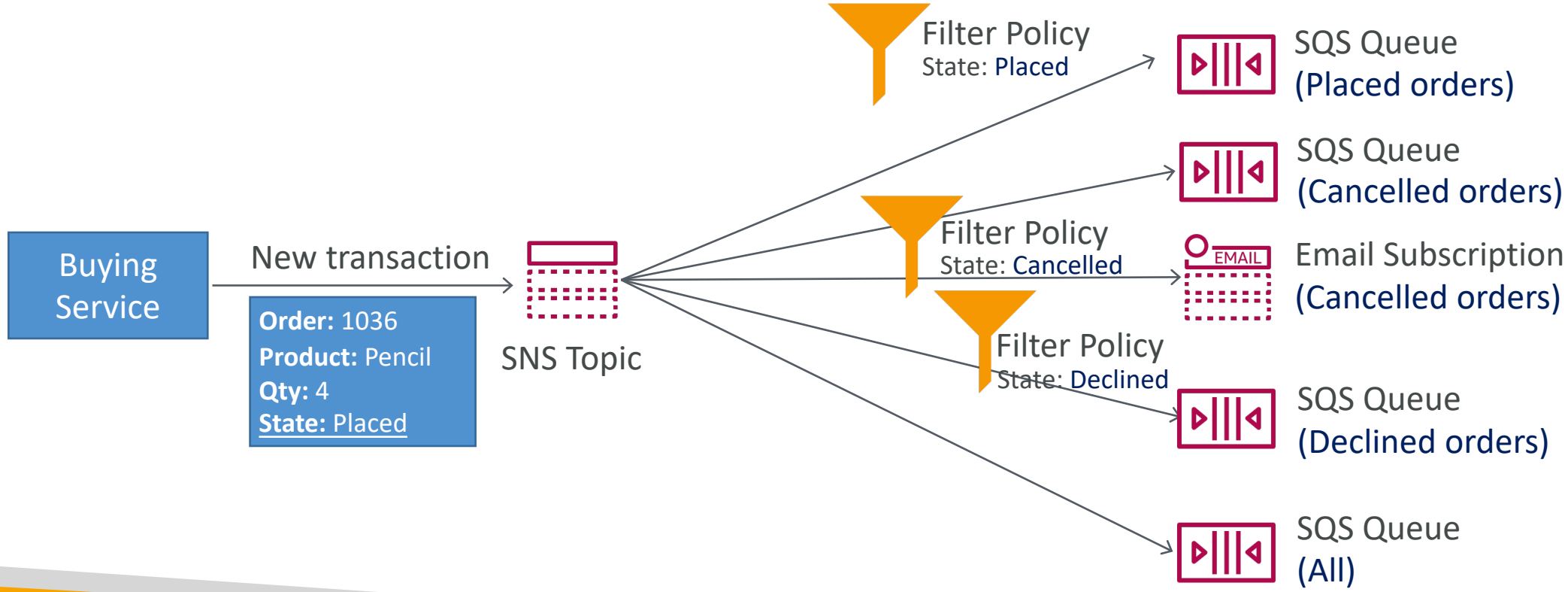


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SNS – Message Filtering

- JSON policy used to filter messages sent to SNS topic's subscriptions
- If a subscription doesn't have a filter policy, it receives every message

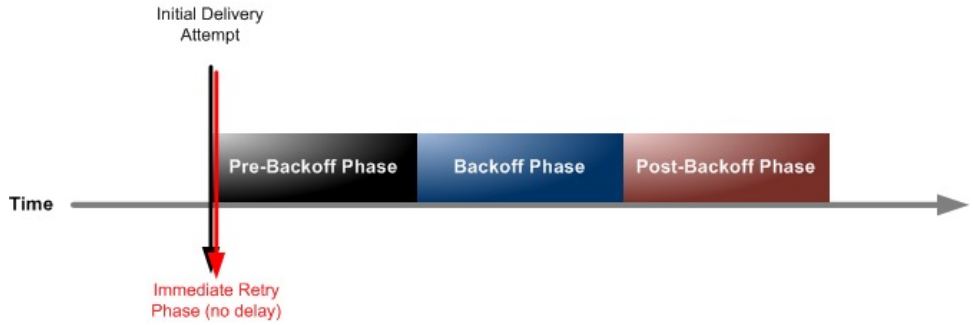


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SNS – Message Delivery Retries

- When a message is delivered to an SNS subscriber, in case of server-side errors, **a delivery policy** is applied



Endpoint type	Delivery protocols	Immediate retry (no delay) phase	Pre-backoff phase	Backoff phase	Post-backoff phase	Total attempts
AWS managed endpoints	Amazon Kinesis Data Firehose AWS Lambda Amazon SQS	3 times, without delay	2 times, 1 second apart	10 times, with exponential backoff, from 1 second to 20 seconds	100,000 times, 20 seconds apart	100,015 times, over 23 days
Customer managed endpoints	SMTP	0 times, without delay	2 times, 10 seconds apart	10 times, with exponential backoff, from 10 seconds to 600 seconds (10 minutes)	38 times, 600 seconds (10 minutes) apart	50 attempts, over 6 hours

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SNS – Custom Delivery Policies

- Only HTTP/S supports custom policies

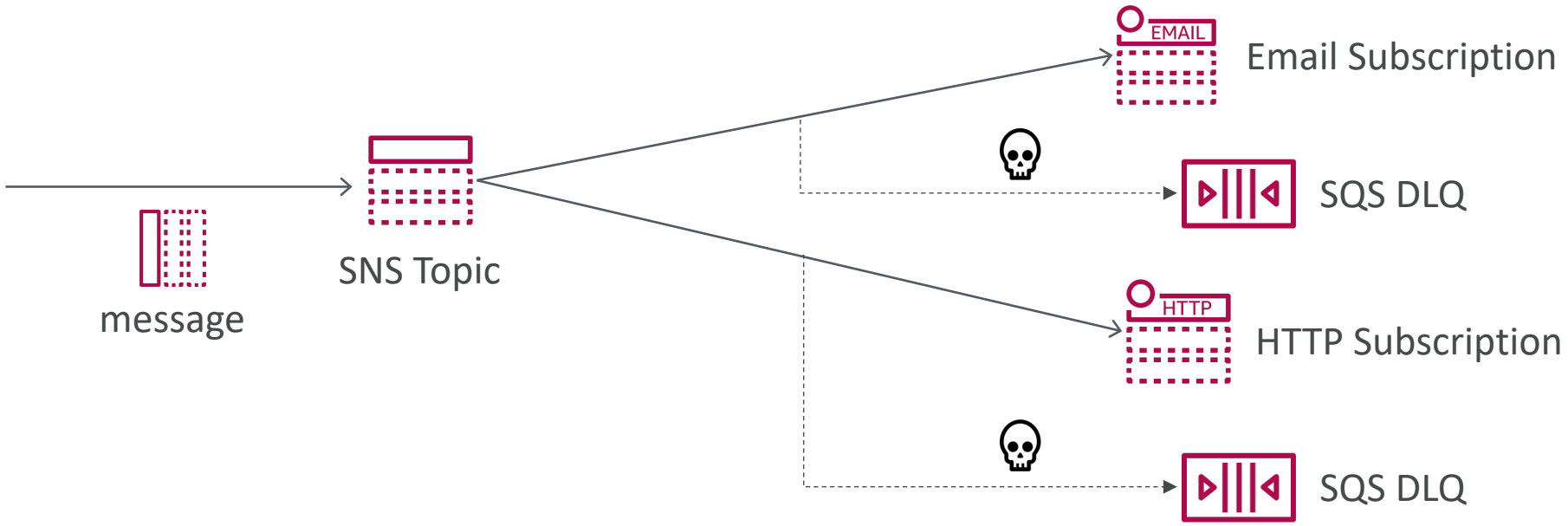
```
{
    "healthyRetryPolicy": {
        "minDelayTarget": 1,
        "maxDelayTarget": 60,
        "numRetries": 50,
        "numNoDelayRetries": 3,
        "numMinDelayRetries": 2,
        "numMaxDelayRetries": 35,
        "backoffFunction": "exponential"
    },
    "sicklyRetryPolicy": null,
    "throttlePolicy": {
        "maxReceivesPerSecond": 10
    }
}
```

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

SNS – Dead Letter Queues

- After exhausting the delivery policy, messages that haven't been delivered are discarded unless you set a DLQ (Dead Letter Queue)
- DLQ are Amazon SQS queues or Amazon SQS FIFO queues (for SNS FIFO)
- Dead Letter Queues are attached to a subscription (rather than a topic)



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com